

- i Write your answers in Inspira. You can of course use separate sheets of paper to draft and experiment, but only the answers in Inspira will be graded. The answers can and should be short.**

Python interpreters such as Self Practice are not available. Hence you have to reason about the answers on your own. Using pencil and paper is a good aid for this.

You can also have with you an A4 paper, hand-written on both sides, with whatever information you want to have available

You will need 24 points out of 60 in this exam to get accepted with grade 3, 36 points for grade 4, and 48 points for grade 5. The exam grade will also be the final grade for the course.

If used as a re-exam for DAT515 or DIT515 from earlier years, the final grade of the course will be the grade from your labs, provided that you get at least 24 points in this exam.

i For your reference: the syntax of (the relevant parts of) Python

```

<stm> ::= <decorator>* class <name> (<name>,*): <block>
| <decorator>* def <name> (<arg>,*): <block>
| import <name> <asname>?
| from <name> import <imports>
| <exp>,* = <exp>,*
| <exp> <assignop> <exp>
| for <name> in <exp>: <block>
| <exp>
| return <exp>,*
| yield <exp>,*
| if <exp>: <block> <elses>?
| while <exp>: <block>
| pass
| break
| continue
| try: <block> <except>* <elses> <finally>?
| assert <exp> ,<exp>?
| raise <name>
| with <exp> as <name>: <block>

<decorator> ::= @ <exp>
<asname>    ::= as <name>
imports     ::= * | <name>,*
<elses>     ::= <elif>* else: <block>
<elif>      ::= elif exp: <block>
<except>    ::= except <name>: <block>
<finally>   ::= finally: <block>
<block>     ::= <stm> <stm>*
<exp> ::= <exp> <op> <exp>
| <name>.<?><name>(<arg>,* )
| <literal>
| <name>
| ( <exp>,* )
| [ <exp>,* ]
| { <exp>,* }
| <exp>[<exp>]
| <exp>[<slice>,*]
| lambda <name>*: <exp>
| { <keyvalue>,* }
| ( <exp> for <name> in <exp> <cond>? )
| [ <exp> for <name> in <exp> <cond>? ]
| { <exp> for <name> in <exp> <cond>? }
| { <exp>: <exp> for <name> in <exp> <cond>? }
| - <exp>
| ~ <exp>
| not <exp>
| <name> := <exp>

```

<keyvalue> ::= <exp>: <exp>

<arg> ::= <name>

| <name> = <exp>

| *<name>

| **<name>

<cond> ::= if <exp>

<op> ::= + | - | * | ** | / | // | % | @ | == | > | >= | < | <= | != | in | not in | and | or

| & | || | ^ | << | >> | :=

<assignop> ::= += | -= | *= | /= | %= | &= | |= | ^=

<slice> ::= <exp>? :<exp>? <step>?

<step> ::= :<exp>?

- 1 Question 1 (16 p). What is the value and its type of the following expressions? Remember that None is also a value! It can also happen that the expression contains an error. In that case, indicate the kind of error, one of `SyntaxError`, `TypeError`, `AttributeError`, `ZeroDivisionError`, `KeyError`, `NameError`, `IndexError`, and explain (in your own words) the reason of the error. (1+1 p for each part)**

`[4,5,6].append(7)`

(Value and Type) or (Error and Explanation)

`{ m: max(2, m) for m in range(1,4) }`

(Value and Type) or (Error and Explanation)

`set("hello")`

(Value and Type) or (Error and Explanation)

`(lambda x: x.x())(True)`

(Value and Type) or (Error and Explanation)

`set() == []`

(Value and Type) or (Error and Explanation)

`{1,2} is {2,1}`

(Value and Type) or (Error and Explanation)

`len({'cat','dog'}.add('fish'))`

(Value and Type) or (Error and Explanation)

`[c for c in "hello" if c != 'l']`

(Value and Type) or (Error and Explanation)

--

Maximum marks: 16

2 Question 2 (12 p). In Lab 1 of this course, we built dictionaries of tram stops, tram lines, and transition times. They were collected into a single dictionary of the following shape:

```
tramnetwork = {
    "stops": {
        "Östra Sjukhuset": {
            "lat": 57.7224618,
            "lon": 12.0478166
        },
        # the positions of every stop
    },
    "lines": {
        "1": [
            "Östra Sjukhuset",
            "Tingvallsvägen",
            # and the rest of the stops along line 1
        ],
        # the sequence of stops on every line
    },
    "times": {
        "Östra Sjukhuset": {},
        "Tingvallsvägen": {
            "Östra Sjukhuset": 1
        },
        # the times from each stop to its alphabetically later neighbours
    }
}
```

2.1 (6p)

Using this definition, write a Python expression whose value is the name (number) of the line which has the largest number of stops.

2.2 (6p)

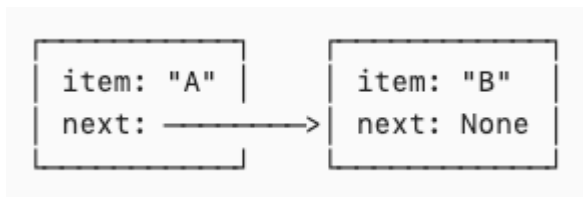
Using the same definition, write a Python expression whose value is set of all the end stops in the network (i.e. the first and last stops of every line). (6p)

Maximum marks: 12

3 Question 3 (16 p). Python has inbuilt support for lists, but we can also implement list-like behaviour using only objects of the following class:

```
class Lst():
    def __init__(self):
        self.item = None
        self.next = None
    def append(self, x):
        if not self.item:
            self.item = x
        else:
            if not self.next:
                self.next = Lst()
            self.next.append(x)
```

Given this class, a list containing the strings "A" and "B" can be represented as two linked objects like so:



Note that this implementation supports empty lists, but cannot store None as a value.

3.1 (3p)

Now, show how you can use the Lst class to create an object lst representing a list containing three values: 42, "foo", and True.

Write the Python code here.

3.2 (3p)

Next, consider the following expression for building a list with two items. Will this expression evaluate correctly (without error)? If yes, what is its type and value? If not, explain what changes could be made in the definition of Lst to make this possible.

Lst().append(4).append(4)

3.3 (6p)

Implement a method items() which allows us to iterate over the items in a Lst object like so (assuming lst is defined as above):

```
>>> for i in lst.items(): print(i)
42
foo
True
```

Define items(self) as a method of class Lst.

3.4 (4p)

As the last task in Question 3, define the method in the Lst class needed for it to work with Python's built-in `len()` function, returning the length of the list (assuming `lst` is defined as above):

```
>>> len(lst)
```

```
3
```

Define a method of class Lst for returning its length.

Maximum marks: 16

- 4 Question 4 (16 p). Below is the API for a (fictitious) Python library for making HTTP requests. All answers in this section should make use of this library. For full marks, you must make sure to respect the documented types. They are given as type hints of form:**

x: <type>

for variables and:

```
def f (... x: <type>...) -> <type>
```

for functions. A <type> expression with a bar T | U means that both T and U are permissible types. dict[T, U] means a dictionary with keys of type T and values of type U.

An HTTP header

```
class Header:
```

```
    key: str
```

```
    value: str
```

An HTTP response

```
class Response:
```

```
    # status code as a number
```

```
    status: int
```

```
    # text explanation of status code
```

```
    status_message: str
```

```
    # a list of Header objects
```

```
    headers: list[Header]
```

```
    # response payload as a raw string (or None)
```

```
    body: str | None
```

```
    # read payload as a JSON string and return dict (or raise JSONDecodeError)
```

```
    def json(self) -> dict
```

An HTTP request

```
class Request:
```

```
    # URL to send request to
```

```
    url: str
```

```
    # HTTP method as string (e.g. GET, POST)
```

```
    method: str
```

```
    # a list of Header objects
```

```
    headers: list[Header]
```

```
    # request payload (i.e. the actual data sent) as string (or None)
```

```
    body: str | None
```

```
    # send request and return Response object
```

```
    def send(self) -> Response
```

```

# Construct and return a Request object without sending it
# headers is a dictionary of strings or None (default None)
# body is a string or None (default None)
def make_request(
    url: str,
    method: str,
    headers: dict[str, str] | None = None,
    body: str | None = None
) -> Request

# Send a request, returning a Response object
# headers is a dictionary of strings or None (default None)
# body is a string or None (default None)
def send_request(
    url: str,
    method: str,
    headers: dict[str, str] | None = None,
    body: str | None = None
) -> Response

# Add a header to a Request object (update in-place, return None)
def add_header(
    req: Request,
    key: str,
    value: str,
) -> None

# Get header value corresponding to key in a Request or Response object (or None)
def get_header(
    r: Request | Response,
    key: str
) -> str | None

```

4.1 (4p)

Your first task is to write code to send a GET request to URL `http://www.example.com/tasks` with header `TaskId: 123`.

Set a local variable `ok` to `True` if the returned response has status code 200, and `False` otherwise.

Write the Python code here.

4.2 (4p)

Assume your response in 4.1 is stored in variable `resp` and its body is the following JSON string:

```
{"id": 123, "desc": "Take exam", "status": ""}
```

Create a POST request (without sending it) to the same URL where the body is the same JSON string as above, but with the status value set to `"done"`. Store the unsent request in variable `req`.

Write the Python code here.

4.3 (4 p)

Assume your request from 4.2 is stored in the variable `req`. Set header `Cache: False` in your request and then send it. Check if you get a header with key `CacheStatus` in the response, and if so print it.

Write the Python code here.

4.4 (4 p)

Add a function `bulk_get(url, n)` which sends `n` individual GET requests to `url` and returns all the responses. Each request should contain a unique header with the key `RequestNum` and a value corresponding to the request number (1..`n`).

Write the Python code here.

Maximum marks: 16